আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম

الجامعة الإسلامية العالمية شيتاغونغ

International Islamic University Chittagong

Lab Report

COURSE CODE : EEE- 4702

COURSE TITLE : Control System I Sessional

EXPERIMENT NO : 01

EXPERIMENT NAME : Mathematical Modeling of a System using MATLAB.

SUBMITTED TO : Mohammed Sadman Kabir ,
Adjunct lecturer, Dept of EEE, IIUC

SUBMITTED BY :

NAME : Mohammad Siam

ID : ET-231096

SEMESTER : 7th

SECTION : 7C

SESSION : SPRING 2026

DEPARTMENT : EEE

DATE OF EXPERIMENT : - - 2026

DATE OF SUBMISSION : 31 - 03 - 2026

REMARK:

1. Objective:

- To understand the concept of Partial Fraction Expansion (PFE).
- To determine residues, poles, and direct terms using MATLAB.
- To apply the residue function for solving Laplace domain expressions.
- To compare theoretical transfer function with MATLAB results.
- To analyze system behavior from poles and zeros.

2. Apparatus / Utility Software:

- Computer or Laptop
- MATLAB Software
- Control System Toolbox in MATLAB
- Calculator (for manual verification)

3. Description of Task :

In this experiment, we worked on Partial Fraction Expansion using MATLAB.
At first, we studied how the residue function works. This function takes numerator and denominator polynomial coefficients and gives three outputs:

- Residues (r)
- Poles (p)
- Direct terms (k)

Task 1 Description:

A transfer function G(s)G(s)G(s) is given which contains multiple polynomial terms in numerator and denominator.
The numerator is formed using:

- (s+ID)(s + ID)(s+ID)
- (s+ID+10)(s + ID + 10)(s+ID+10)

The denominator consists of:

- s
- $(s + 30)$
- $(s + ID + 5)$
- $(s^2 + 3s + 30)$
- $(s^2 + 5s + ID + 15)$

All polynomials are combined using convolution to get a single numerator and denominator.
Then:

- Transfer function is created using tf()
- Zeros and poles are calculated using roots()
- Partial Fraction Expansion is obtained using residue()

Task 2 Description:

In this task, a simple electrical circuit is considered.
The transfer function is derived theoretically as:

$$G(s) = \frac{Ls}{LCs^2 + RCs + 1}$$

Then values are substituted:

- $R = 20 + ID$
- $C = 1 + ID$
- $L = 2 + ID$

After substituting values:

- Transfer function is formed
- Poles and zeros are calculated
- PFE is performed using MATLAB

4. MATLAB Code and result:

Task :1

EEE4702LAB2.m *

```
clc
clear
close all

% ID= ET231096
ID = 96;
num1 = [1 ID];          % (s + ID)
num2 = [1 ID+10];       % (s + ID + 10)

numg = 103 * conv(num1 , num2);

d1 = [1 0];             % s
d2 = [1 30];            % (s + 30)
d3 = [1 ID+5];          % (s + ID + 5)
d4 = [1 3 30];          % (s^2 + 3s + 30)
d5 = [1 5 ID+15];       % (s^2 + 5s + ID + 15)

deng = conv(conv(conv(conv(d1,d2),d3),d4),d5);

G = tf(numg , deng)

disp('Transfer Function G(s):')
G
disp('Zeros of the system:')
z = roots(numg)

disp('Poles of the system:')
p = roots(deng)

[r , p , k] = residue(numg , deng);
disp('Residues:')
r

disp('Poles:')
p

disp('Direct Polynomial Term:')
k
```

```
G =

                         103 s^2 + 20806 s + 1.048e06
  ---------------------------------------------------------------------------
  s^7 + 139 s^6 + 4234 s^5 + 45159 s^4 + 539283 s^3 + 1.9e06 s^2 + 1.009e07 s

Continuous-time transfer function.
Model Properties
Transfer Function G(s):

G =

                         103 s^2 + 20806 s + 1.048e06
  ---------------------------------------------------------------------------
  s^7 + 139 s^6 + 4234 s^5 + 45159 s^4 + 539283 s^3 + 1.9e06 s^2 + 1.009e07 s

Continuous-time transfer function.
Model Properties
Zeros of the system:

z =

  -106
   -96

Poles of the system:

p =

   1.0e+02 *

   0.0000 + 0.0000i
  -1.0100 + 0.0000i
  -0.3000 + 0.0000i
  -0.0250 + 0.1023i
  -0.0250 - 0.1023i
  -0.0150 + 0.0527i
  -0.0150 - 0.0527i

Residues:

r =

  -0.0000 + 0.0000i
```

```
Residues:

r =

  -0.0000 + 0.0000i
  -0.0003 + 0.0000i
   0.0149 - 0.0140i
   0.0149 + 0.0140i
  -0.0667 + 0.0401i
  -0.0667 - 0.0401i
   0.1039 + 0.0000i

Poles:

p =

   1.0e+02 *

  -1.0100 + 0.0000i
  -0.3000 + 0.0000i
  -0.0250 + 0.1023i
  -0.0250 - 0.1023i
  -0.0150 + 0.0527i
  -0.0150 - 0.0527i
   0.0000 + 0.0000i

Direct Polynomial Term:

k =

     []
```

Task :2

```
% Control System I Lab 2 task 2 : Partial Fraction Expansion

clc;
clear;
close all;

%% Student ID = ET231096
id_val = 96;

R = 20 + id_val;
C = 1 + id_val;
L = 2 + id_val;

fprintf('Component Values:\n');
fprintf('R = %d Ohm\n',R);
fprintf('C = %d F\n',C);
fprintf('L = %d H\n\n',L);
num_tf = [L 0];                % Numerator (L*s)
den_tf = [L*C R*C 1];          % Denominator (L*C*s^2 + R*C*s + 1)

sys = tf(num_tf,den_tf);

disp('Transfer Function G(s):');
sys

fprintf('\nZeros of the system:\n');
z_sys = roots(num_tf)

fprintf('\nPoles of the system:\n');
p_sys = roots(den_tf)

[res,pole,dir_term] = residue(num_tf,den_tf);

fprintf('\nResidues (r):\n');
res

fprintf('\nPoles from PFE (p):\n');
pole

fprintf('\nDirect term (k):\n');
dir_term
```

```
Component Values:
R = 116 Ohm
C = 97 F
L = 98 H

Transfer Function G(s):

sys =

            98 s
  ----------------------
  9506 s^2 + 11252 s + 1

Continuous-time transfer function.
Model Properties

Zeros of the system:

z_sys =

     0


Poles of the system:

p_sys =

   -1.1836
   -0.0001


Residues (r):

res =

    0.0103
   -0.0000


Poles from PFE (p):

pole =

   -1.1836
   -0.0001


Direct term (k):

dir_term =

     []

fx >>
```

5. Analysis of Each Output:

Task 1 Analysis :

After running the MATLAB code, several outputs are obtained such as transfer function, zeros, poles, residues, and direct terms. Each of them has a specific meaning:

1. Transfer Function G(s)G(s)G(s):

The transfer function is formed by combining all numerator and denominator polynomials using convolution. This gives a higher-order system. The transfer function represents the relationship between input and output in the Laplace domain. Since the denominator has multiple factors, the system becomes higher order and more complex. This type of system usually has multiple modes of response.

2. Zeros of the System:

Zeros are calculated using the roots of the numerator polynomial. These are the values of sss that make the transfer function zero. Physically, zeros affect the shape of the system response, especially the transient behavior. For example, zeros can introduce overshoot or reduce rise time. In this case, since the numerator contains (s+ID)(s + ID)(s+ID) and (s+ID+10)(s + ID + 10)(s+ID+10), the zeros will be negative real values, which indicates stable zero locations.

3. Poles of the System:

Poles are obtained from the denominator polynomial. These are very important because they determine system stability.

- If poles are in the left half of the s-plane (negative real part), the system is stable.
- If any pole lies in the right half, the system becomes unstable.

Since the denominator includes both linear and quadratic terms, we get:

- Real poles from linear factors
- Possibly complex conjugate poles from quadratic factors

Complex poles indicate oscillatory behavior in the system response.

4. Residues (r):

Residues represent the coefficients of each term in the partial fraction expansion. Each residue corresponds to one pole.

This means each term of the form:

$$\frac{r_i}{s - p_i}$$

contributes to the overall system response.

Higher magnitude residues indicate stronger contribution of that pole in the output response. If a residue is very small, that term has very little effect on the system.

5. Poles from PFE (p):

The poles obtained from the residue() function match the roots of the denominator. This confirms that MATLAB is correctly performing the decomposition. Each pole is associated with one exponential term in the time-domain response.

6. Direct Term (k):

In this task, the direct term is empty. This happens because the degree of the numerator is less than the degree of the denominator.

So, there is no polynomial division term, and the system is strictly proper.

7. Overall Interpretation:

Each pole-residue pair corresponds to a term in time domain like:

$$r_i e^{p_i t}$$

So the full response is the sum of multiple exponential terms. Real poles give exponential decay, while complex poles give oscillations.

Task 2 Analysis :

In Task 2, the system is simpler and represents a real electrical circuit.

1. Transfer Function:

The transfer function is:

$$G(s) = \frac{Ls}{LCs^2 + RCs + 1}$$

After substituting values of R,C,L we get a second-order system. This is a standard form used in control systems and electrical circuits.

2. Zero of the System:

The numerator is Ls, so there is one zero at:s=0

This means the output becomes zero when input frequency is zero (DC condition). It also indicates that the system blocks constant signals partially.

3. Poles of the System:
Poles are obtained from the quadratic denominator:

$$LCs^2 + RCs + 1 = 0$$

Depending on the values of R,C,L the poles can be:

- Real and distinct
- Real and repeated
- Complex conjugate

In most cases here, poles are complex conjugate because of large values of RRR and CCC. This indicates oscillatory response with damping.

4. Residues:
Residues are calculated for each pole. These values determine how much each exponential term contributes to the output. For complex poles, residues are also complex, and they combine to form a real oscillatory signal in time domain.

5. Direct Term:
There is no direct term because the numerator degree is less than denominator degree. So, the system is strictly proper.

6. Physical Meaning:
This system behaves like an RLC circuit.

- Inductor (L) controls energy storage in magnetic field
- Capacitor (C) controls energy storage in electric field
- Resistor (R) controls damping

So, poles reflect how fast energy decays and whether oscillation occurs.

7. System Behavior:

- If poles have negative real part → stable system
- If imaginary part exists → oscillation
- Larger resistance → more damping → less oscillation

6. Discussion:
In this experiment, the concept of Partial Fraction Expansion was studied both mathematically and using MATLAB. The use of MATLAB made the calculation much easier, especially for higher-order systems where manual calculation becomes difficult and time-consuming. From Task 1, it was observed that complex transfer functions can be simplified into smaller terms using residues and poles. This helps in understanding how each part of the system contributes to the overall response. The poles obtained from MATLAB confirmed the stability of the system, as most of them were located in the left half of the s-plane. In Task 2, a real-life electrical circuit was analyzed, which made the experiment more practical and meaningful. The relationship between circuit components and system behavior became clear. It was seen that changing values of resistance, capacitance, and inductance directly affects the poles and hence the system response. Overall, the experiment improved understanding of Laplace domain analysis, system stability, and time response. It also showed how powerful MATLAB is for solving engineering problems quickly and accurately.